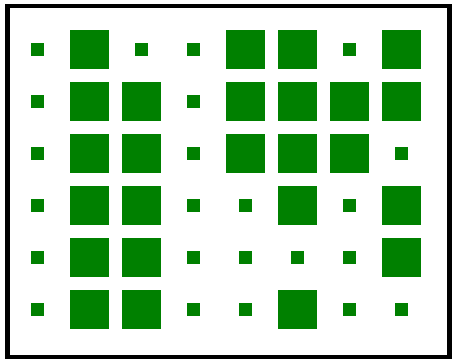


Class 11
Memory Organization and Pointers

CS 330
Programming Languages and Compilers

https://www.wsj.com/tech/ai/yann-lecun-ai-meta-0058b13c

TECHNOLOGY | ARTIFICIAL INTELLIGENCE Follow

He's Been Right About AI for 40 Years. Now He Thinks Everyone Is Wrong.

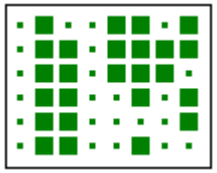
Yann LeCun invented many fundamental components of modern AI. Now he's convinced most in his field have been led astray by the siren song of large language models.



Yann LeCun has become the odd man out at Meta. JUSTIN J WEE FOR WSJ

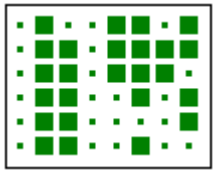
LLM Roundtable

Computer Memory (RAM)



- RAM is managed by the OS
  - Each program is provided with memory in a way that it believes it owns it all
  - Sequential addresses from 0 to the number of bytes available
  - For each process (running program), logically divided into 4 segments
    - Code
    - Data
    - Heap
    - Stack

Computer Memory (RAM)



- Code: machine code (compiled/linked)
- Data: constants and global variables
- Heap: dynamically allocated (malloc/free/new)
- Stack: local variables, function calls, params
- Attempt to access memory outside the correct segment: *segmentation fault*

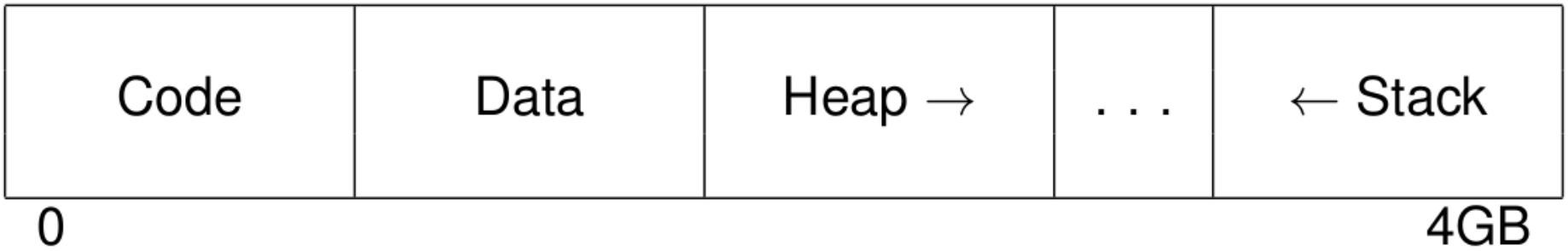
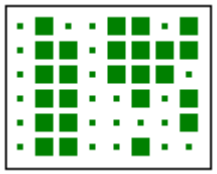


Figure 9.2: Logical Segments

Computer Memory (RAM)



- Heap grows up: memory addresses increase
- Stack grows down: memory addresses decrease
- Guard page below stack identifies end of memory available for stack

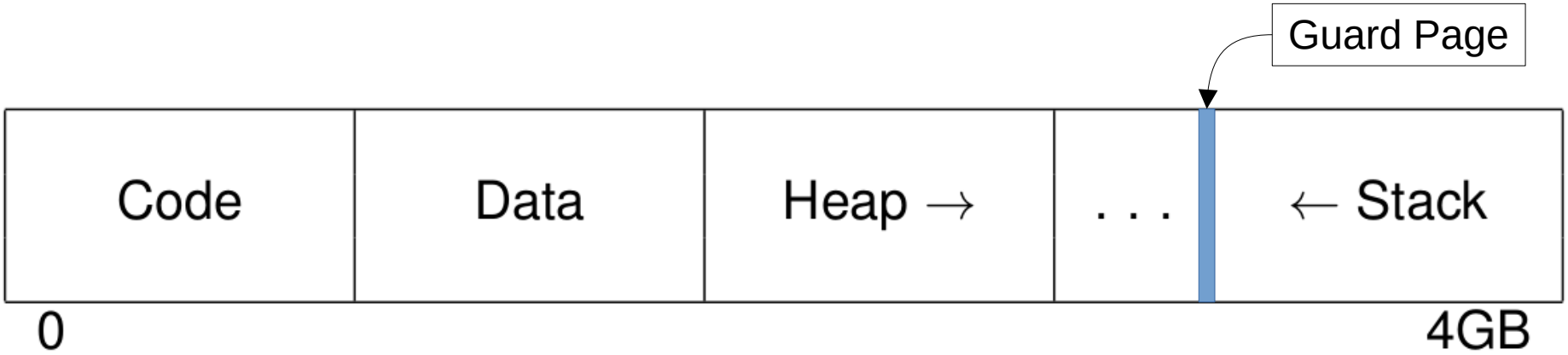
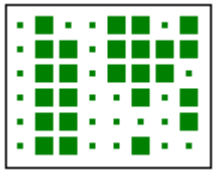


Figure 9.2: Logical Segments

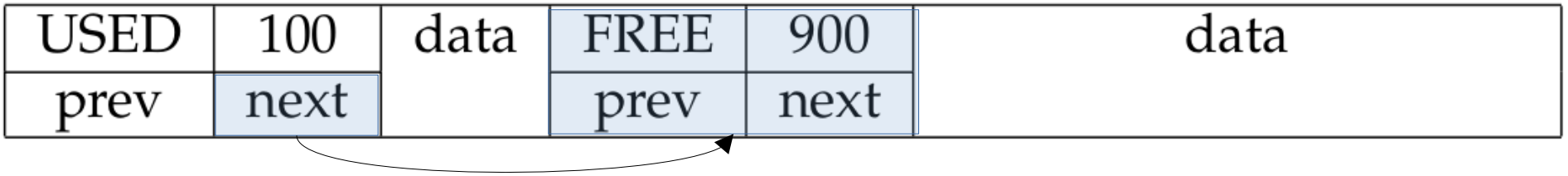
Allocating from the Heap



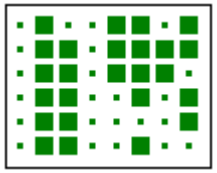
- In C, `malloc()` is used to allocate from the heap
- Java uses the keyword `new`, as does C++
- Heap is often managed using a list



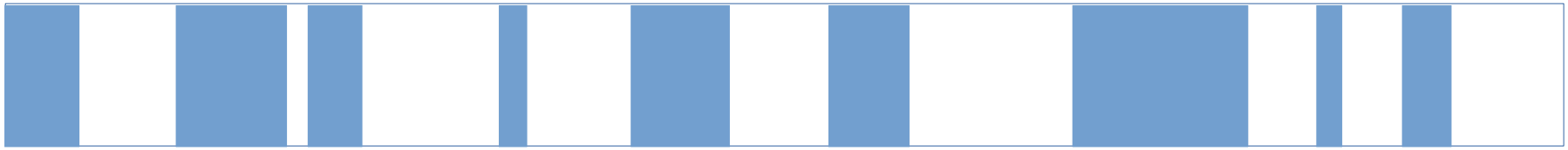
malloc(100)



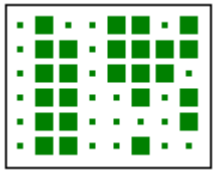
Memory Fragmentation



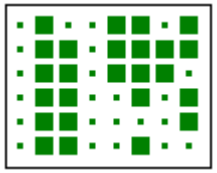
- Since allocations of heap memory are arbitrarily sized and freeing is not ordered, the memory becomes fragmented with free and allocated sections scattered about
  - Could lead to a situation where there is enough total free memory for an allocation, but it is not contiguous, so the allocation fails



Heap Allocation Schemes

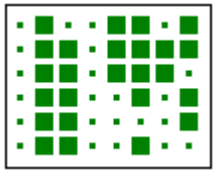


- **Best Fit**: search the heap for the smallest available fragment that is large enough
- **Worst Fit**: search the heap for the largest available fragment that is big enough
  - Reduces small fragments
- **First Fit**: Find the first fragment that is big enough
- **Next Fit**: Starting from the last allocation, find the first fragment that is big enough
  - Performs well in the general case



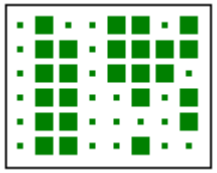
Memory Pointers

Pointer Type Overview



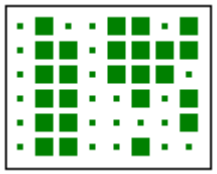
- **A data type**
- Store memory addresses or nil (no address)
- Supports indirect addressing
- Used for dynamic storage management
  - Can access data at a particular memory address in the heap
- Provides a mechanism for data structures that shrink and grow during program execution

Using Pointers



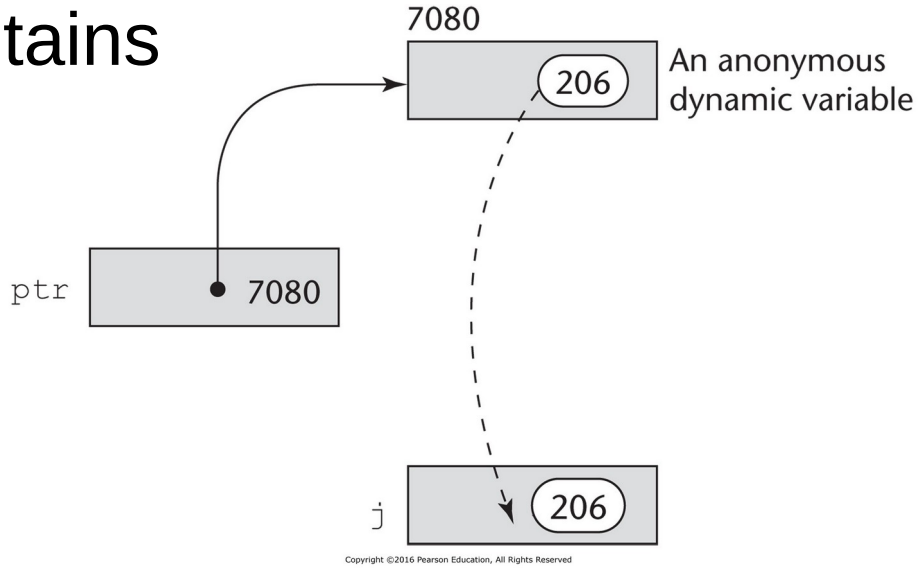
- **Pointer reference** (the address)
- **Indirect reference** (a.k.a. dereference) access the data which is in the memory address stored in the pointer
  - Dereferencing is usually explicit (C, C++ with the * operator) but may be implicit (e.g. Ada)

Dereferencing Example

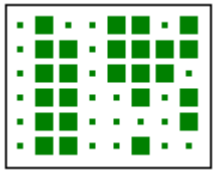


```
int *ptr = (int *) malloc(sizeof(int));
*ptr = 206;
int j = *ptr;
```

- Variable `ptr` is a pointer and contains the memory address 7080
- Memory address 7080 contains the integer value 206
- j is an `int` variable
- j is assigned the value that `ptr` is pointing to

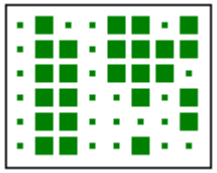


Pointer Problems: Dangling Pointer



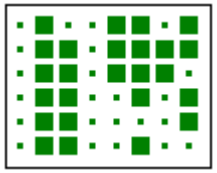
- **Dangling pointer** (dangling reference in those languages that don't have pointers)
  - Pointer contains deallocated address
    - p1 points to heap dynamic variable
    - deallocate p1's heap dynamic variable
    - p1 is a dangling pointer to the deallocated heap dynamic variable
- If language does not permit explicit deallocation of heap dynamic variables, then there is no possibility of dangling pointers

Pointer Problems: Memory Leak



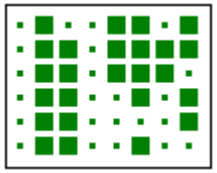
- Changing or losing a pointer's value (assigned a different address or goes out of scope) before deallocating the memory it points to results in a **memory leak**
- Let’s depict some dangling pointer and memory leakage problems on the board

Pointer Design Decisions: C and C++



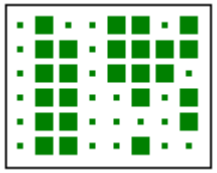
- Dangling pointer and memory leakage risks exist in these languages
- Programmer must manage memory properly to prevent these issues
- One use of pointers is to pass variables by reference (so they may be changed within the function)
  - Contrast with pass by value, where the variable's value is copied to new temporary variable (parameter)

Fix Dangling Ptr: Tombstones



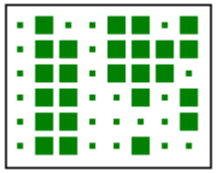
- A **tombstone** points to (holds the address of) where the data is (a heap-dynamic variable)
- Pointers can only point to a tombstone (which in turn points to the actual data)
- When deallocated, the tombstone is set to nil
  - What does this solve?
- Why do you think they aren't used in popular languages that have dangling pointer issues?

Fix Dangling Ptr: Locks-and-keys (1/3)



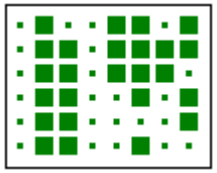
- Pointers and variables need different implementations for this method
- Pointers are ordered pairs of an **integer key** and a **memory address**
- Heap-dynamic variables made up of:
  - Header cell that stores **integer lock** value
  - Storage cell(s) for the variable itself

Fix Dangling Ptr: Locks-and-keys (2/3)



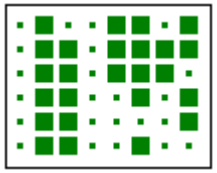
- During allocation a lock value is calculated and placed in the *key* portion of the pointer and in the *lock* cell of the variable
- Key and lock cells are compared when the pointer is dereferenced and if they're the same it's a legal reference otherwise it is an illegal reference (generating a run-time error)

Fix Dangling Ptr: Locks-and-keys (3/3)



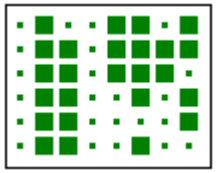
- Multiple pointers may point to the variable but they all must have the same key
- When variable is deallocated (explicitly), the variable's header cell is changed to an illegal lock value (no key could match it)

Fix Memory Leak: Reference Counter



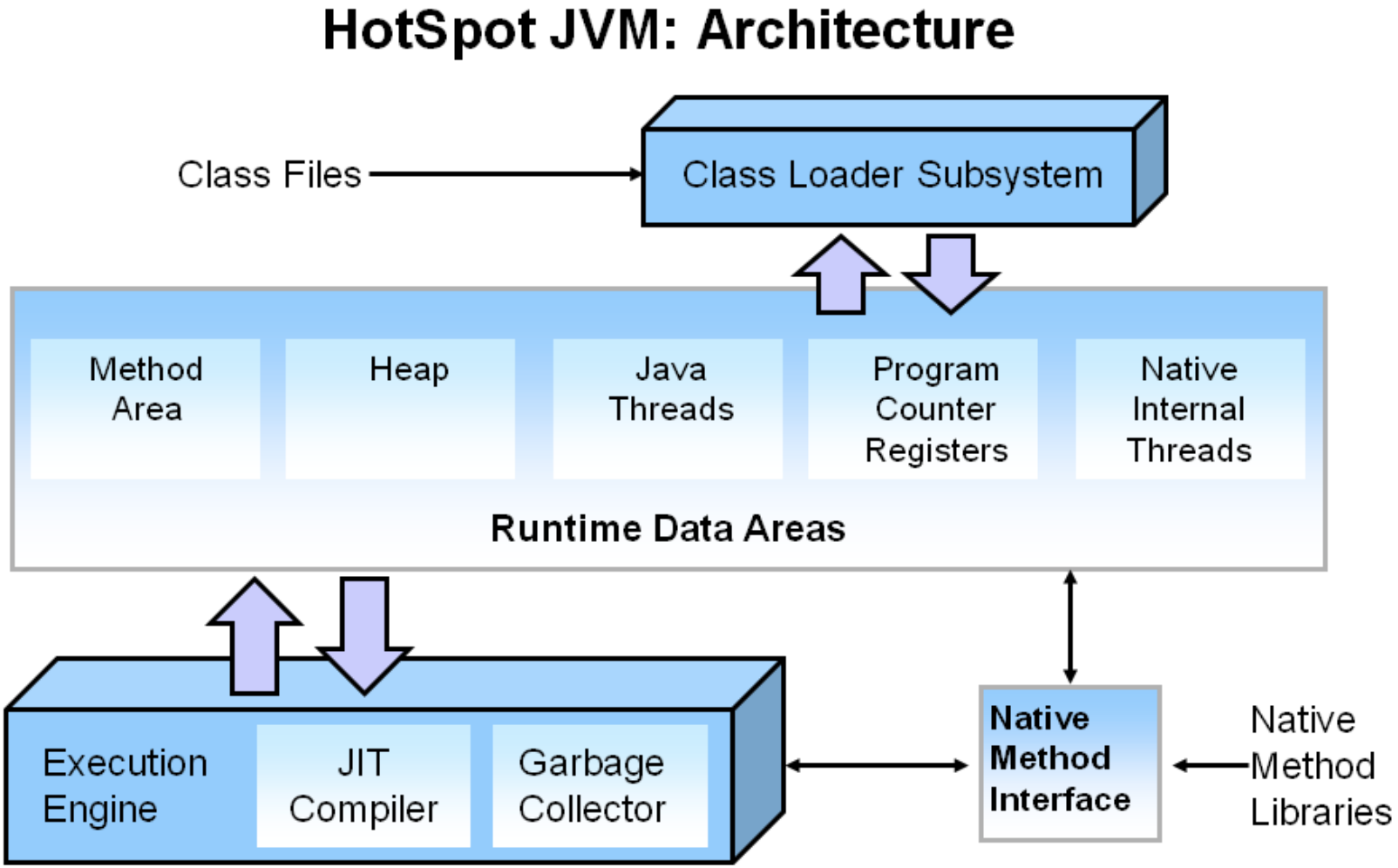
- Store **reference counter** for memory cell
  - Value is the current number of pointers to it
  - When pointers change value or are destroyed, the counter is decremented. It is also checked to see if it is zero. If it is, then the memory can be reclaimed.
  - Any drawbacks to this method?
    - Space
    - Time
    - Circular references

Fix Memory Leak: Garbage Collection



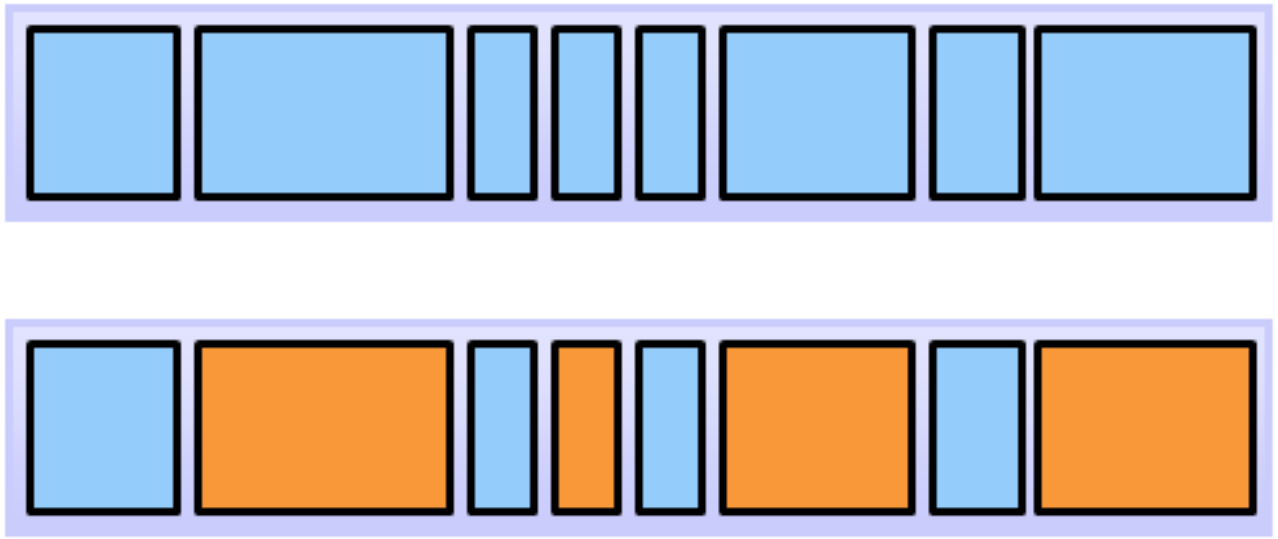
- Every allocated heap cell needs a flag to indicate whether or not its is in use
- To reclaim memory, the garbage collector:
  1) Set all the flags as “memory not in use”
  2) Follows variables to find memory that is being used, marking those flags as “memory in use”
  3) When scan completes, all cells with flags marked “memory not in use” are freed
- Process complicated by allocations creating memory areas of different sizes

JVM Garbage Collection: Overview



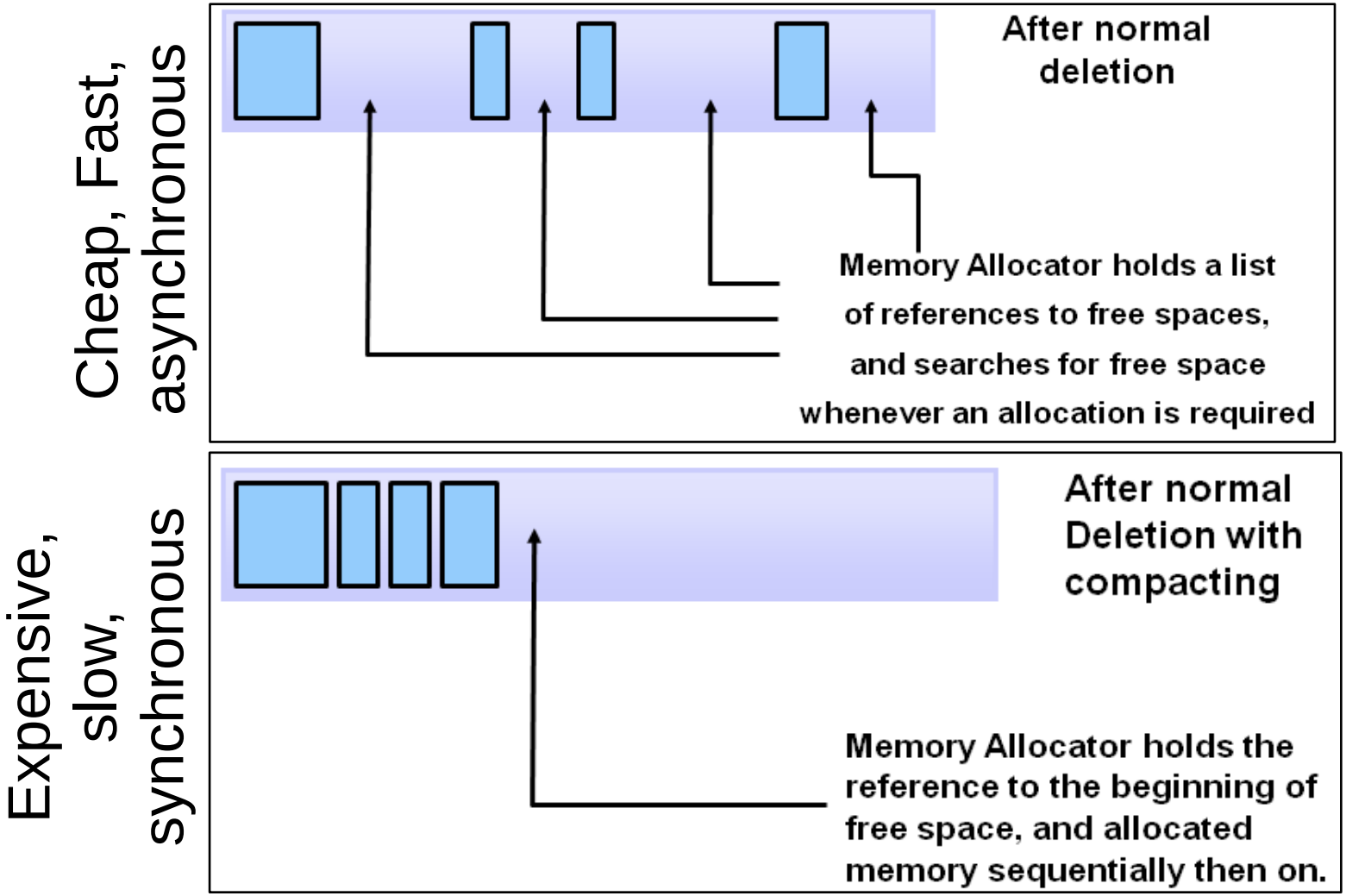
https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html

Marking Phase



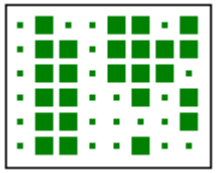
https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html

Sweep/Delete Options



https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html

Homework



- Due Sunday night (10/11)
  - **Journal** entry for week 5
    - 2-1 rule (minimum 2 paragraphs, 1 question)
- Monday (10/12)
  - No class and no office hours (Study Day)
- Wednesday (10/14)
  - **Lab** 3 due (scanner, lexical analyzer)
- Due before Friday's class (10/16)
  - LLM: Developer survey... Perusall